

Advanced SQL for Data Engineer

Course Overview

If there is a shortcut to becoming a Data Scientist, then learning to think and work like a In this course you will learn SQL inside out- from the very basics of Select statements to advanced concepts like JOINS.

You will:

- write foundational SQL statements like: SELECT, INSERT, UPDATE, and DELETE
- filter result sets, use WHERE, COUNT, DISTINCT, and LIMIT clauses.
- differentiate between DML & DDL
- CREATE, ALTER, DROP and load tables.

Contents

1. The Different Types of Table Joins
2. Nested Queries
3. Creating and Saving Views Queries
4. SQL Functions
5. Creating Stored Procedures
6. Working with Triggers
7. User-Defined Functions (UDFs)
8. Window Functions
9. Common Table Expressions (CTEs)
10. Indexes
11. Cursors

Types of Join

- **Equi JOIN:** Primary key and Foreign key equality.
- **Non-Equi JOIN:** When there's a relation between two or more tables but there's no PK and FK relation between them.

Equijoins and Non-Equijoins

- Equijoin: when the join condition is based on the equality operator (Ex. $D.dno = e.deptno$)
- Non-Equijoin: when the join condition is based on any operator rather than the equality operator

Types of Equi Join

- **INNER JOIN:** Returns rows when there is at least one match in both tables
- **Outer JOIN:** Returns matched rows as well as rows when there is no match in one of the tables
- **Self JOIN:** Write the same table many times in the FROM and join itself
- **Cartesian Product:** Returns all possible combinations of rows

Join Queries (Inner Join)

Retrieve the name , address of all employees who work for Research Department

```
SELECT    fname , Lname, address
FROM      Employee , Department
WHERE      Dname='research' AND
           Department.number=employee.Dno
```

Ambiguous Column Names

- When you query two tables with the same column name
 - There is a column called “ name “ in both tables.
 - There is a column called “ ID “ in both tables.
- You should prefix the ambiguous column names with table name
- It's generally recommended that you prefix all column names with table name

for better performance

9/19/2024

@MCIT-2024

Example

Query All employees working for any department.

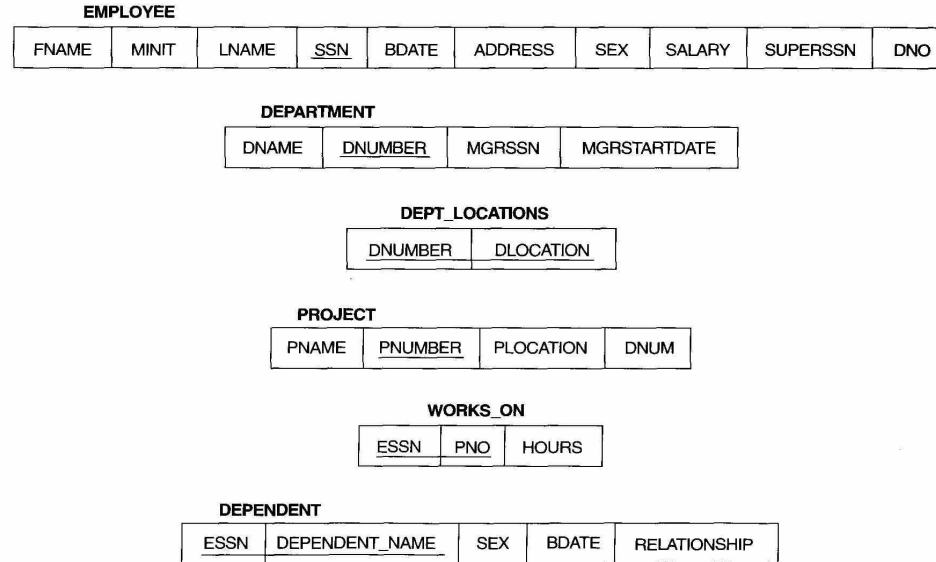


Figure 7.5 Schema diagram for the COMPANY relational database schema; the primary keys are underlined.

Example

- Query All employees working for any department.

```
SELECT department.name, employee.ID, employee.name, Salary  
FROM   department, employee  
WHERE  department.id = employee.deptid  
ORDER BY department.name
```

Table Alias

- You can add alternative name (Alias) for Table name
- Alias can be useful for long or complex table names
- You can use table alias instead of table name to resolve ambiguity

```
SELECT      e.ID , d.name , e.name , e.Salary
FROM        department d , employee e
WHERE       d.id = e.deptid
ORDER BY    d.name
```

Self Join

- Query the employees table to list name of each employee and his supervisor's name.

```
SELECT e.name Employee_name , s.name Supervisor  
FROM employees e , employees s  
WHERE e.supervisorID = s.ID
```

Self Join

ID	Employee_name	SupervisorID
1122	Ahmed Ali	2233
2233	Kamel Mohamed	1234
1234	Hanaa Sobhy	2233
3216	Amr Omran	NULL
9685	Noha Mohamed	2233
5124	Edward Hanna	2233
6699	Mariam Adel	5124
5216	Maged Raoof	9685

Outer Join

- **LEFT JOIN:** Return all rows from the left table, even if there are no matches in the right table
- **RIGHT JOIN:** Return all rows from the right table, even if there are no matches in the left table
- **FULL JOIN:** Return rows when there is a match in one of the tables

Outer Join

- Display all departments' information even if they have no employees assigned.

```
SELECT e.name AS Employee, d.dept_id,  
       d.name AS Department  
FROM   employees e RIGHT OUTER JOIN departments d  
ON     e.dept_id= d.id ;
```

Employees	Dept_ID	Departments
Ahmed Ali	100	IT
Mohamed Samir	200	Marketing
Mona Selim	100	IT
	300	HR

Example (Non-Equijoin)

- Display employees information (name , salary , title) with salary grade for each employee.

```
SELECT e.name, e.salary, j.grade
FROM employees e, job_grades j
WHERE e.salary BETWEEN j.lowsal
AND j.highsal;
```

LowSal	HighSal	Grade
1000	4000	B
4000	7000	A

employees

EmpNo	Name	Salary
100	Ahmed	5000
200	Mai	3000

Sub-Queries

- Sub-Query (Nested Query): is a complete SELECT query inside another SELECT
- The Inner query is executed first then the outer query
- The inner query is usually placed in the WHERE or HAVING clauses
- Sometimes it is placed in the FROM clause and called “inline view”

Examples

- Find the names of employees whose working locations are giza

```
SELECT name
FROM      Employee
WHERE Dno IN (SELECT Dnumber
               FROM dept
               WHERE location='giza')
```

Examples (Cont'd)

- Display department name for the highest paid employee

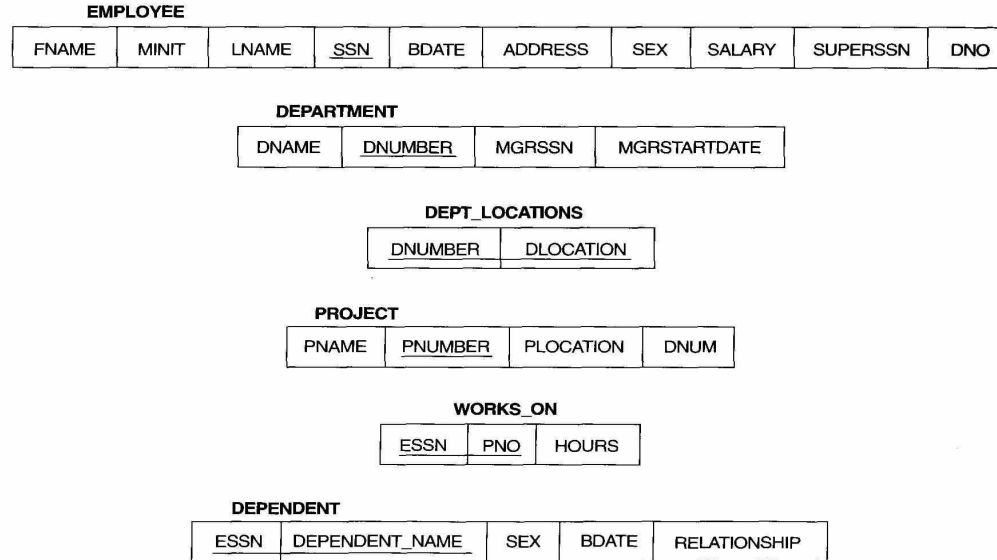


Figure 7.5 Schema diagram for the COMPANY relational database schema; the primary keys are underlined.

Examples (Cont'd)

- Display department name for the highest paid employee

```
SELECT dname
FROM dept
WHERE deptno = (SELECT deptno
                FROM emp
                WHERE sal =
                  (SELECT MAX(sal)
                   FROM EMP));
```

Examples (Cont'd)

- Find the names of employees whose salary is greater than the salary of the employees in department 5

```
SELECT Lname , Fname
FROM employee
WHERE salary > ALL ( SELECT salary
                      FROM employee
                      WHERE Dno=5)
```

Union Operator

- **UNION** operator is one of the set operators that unifies the output of two select statements into one result.
- Two conditions must apply for the two selects
 - Same number of columns
 - Same data type of columns
- The result displays the names of the first query
- If one row is contained in the result of the two selects it is displayed only once.
- **UNION ALL** is functionally equivalent to **UNION** operator, except that it doesn't eliminate repetition in the result set

Examples

- Find the departments numbers whose location in GIZA or manager no = 10

```
SELECT dnumber  
FROM department WHERE MRGSSN= 10
```

Union

```
SELECT dnumber  
FROM dept_locations  
WHERE dlocation='GIZA'
```

One
Result

Examples (Cont'd)

- Display names of current employees as well as previous employees

```
SELECT Name  
FROM Employees  
UNION  
SELECT Name  
FROM Employees_retired
```

Views

- A view is a logical table based on a table or another view
- A view contains no data of its own, but is like a window through which data from tables can be viewed or changed
- The tables on which a view is based are called *base tables*
- The view is stored as a `SELECT` statement in the data dictionary.

Advantages of Views

- Restrict data access
- Make complex queries easy
- Provide data independence
- Present different views of the same data

Simple Views and Complex Views

Feature	Simple Views	Complex Views
Number of tables	One	One or more
Contain functions	No	Yes
Contain groups of data	No	Yes
DML operations through a view	Yes	Not always

- DML operations can be performed on simple views, however for complex views they are not always applicable

Creating Views

```
CREATE VIEW view [ (column 1 [ , column2 ] ... ) ]  
AS subquery  
[ With Check Option ] ;
```

Creating Views (Cont'd)

- Create a view to display employee names and total hours employee worked on a project

```
CREATE VIEW vw_work_hrs  
AS  
SELECT Fname , Lname , Pname , Hours  
FROM Employee, Project , Works_on  
WHERE SSN=ESSN AND PNO=PNUMBER
```

Retrieving Data from a View

- SELECT fname, lname, hours

FROM vw_work_hrs

- SELECT *

FROM vw_work_hrs

Views with Check option

CREATE VIEW Suppliers

AS

SELECT *

FROM suppliers

WHERE status > 15

WITH CHECK OPTION;

Modifying a View

- Syntax

```
CREATE OR REPLACE VIEW view_name  
AS  
Sub-query
```

- Example

```
CREATE OR REPLACE VIEW vw_work_hrs  
AS  
SELECT Fname , Lname , Pname , Hours  
FROM Employee, Project , Works_on  
WHERE SSN=ESSN AND PNO=PNUMBER AND Dno = 5;
```

Removing a View

- Syntax

```
DROP VIEW view_name;
```

- Example

```
DROP VIEW vw_work_hrs
```

SQL Functions

- **Scalar Functions:** Operate on a single value and return a single value.
- **Aggregate Functions:** Compute a single result from multiple rows.
- **Mathematical Functions:** Perform arithmetic operations.
- **String Functions:** Manipulate text.
- **Date & Time Functions:** Work with date/time values.

Scalar Functions

- Use Case:

Standardizing text data by converting all employee names to uppercase.

- Example:

```
SELECT UPPER(name) FROM employees;
```

Aggregate Functions

- Use Case:

Counting the total number of orders placed.

- Example:

```
SELECT COUNT(*) FROM orders;
```

Mathematical Functions

- Use Case:

Rounding off the average salary to two decimal places.

- Example:

```
SELECT ROUND(AVG(salary), 2) FROM employees;
```

String Functions

- Use Case:

Creating a full name column from separate first and last name columns..

- Example:

```
SELECT CONCAT(first_name, ' ', last_name) FROM employees;
```

Date, Time Functions

Most databases contain special datatypes for dates and times

DATE:

YYYYMMDD

TIME:

HHMMSS

TIMESTAMP:

YYYYXXDDHHMMSSZZZZZZ

Date / Time functions:

YEAR(), MONTH(), DAY(), DAYOFMONTH(), DAYOFWEEK(),
DAYOFYEAR(), WEEK(), HOUR(), MINUTE(), SECOND()

Date, Time Functions (continued)

Example 11: Extract the DAY portion from a date:

```
select DAY(RESCUEDATE) from PETRESCUE  
where ANIMAL='Cat'
```

Example 11: Results:

ID	ANIMAL	QUANTITY	COST	RESCUEDATE
1	Cat	9	450.09	2018-05-29
7	Cat	1	44.44	2018-06-11

29

11

Date or Time Arithmetic

Example 13: What date is it 3 days after each rescue date?

```
Select DATE_ADD(RESCUEDATE, INTERVAL 3 DAY) from PETRESCUE
```

Example 13: Results:

	ID	ANIMAL	QUANTITY	COST	RESCUEDATE	+ 3 DAYS
2018-06-01	1	Cat	9	450.09	2018-05-29	2018-06-01
2018-06-04	2	Dog	3	666.66	2018-06-01	2018-06-04
2018-06-07	3	Dog	1	100	2018-06-04	2018-06-07
2018-06-07	4	Parrot	2	50	2018-06-04	2018-06-07
2018-06-13	5	Dog	1	75.75	2018-06-10	2018-06-13

Date or Time Arithmetic

Special Registers:

CURRENT_DATE, CURRENT_TIME

Example 14: Find how many days have passed since each RESCUEDATE till now:

```
Select FROM_DAYS(DATEDIFF(CURRENT_DATE, RESCUEDATE)) from PETRESCUE
```

Example 14: Sample result (format YYYY-MM-DD):

0001-09-21

What is a Stored Procedure?

A set of SQL statements stored and executed on the database server:

- Write in many different languages
- Accept information in the form of parameters
- Return results to the client

Benefits of Stored Procedures

- Reduction in network traffic
- Improvement in performance
- Reuse of code
- Increase in security

CREATE PROCEDURE statement

```
DELIMITER $$  
CREATE PROCEDURE UPDATE_SAL (IN empNum CHAR(6), IN rating  
SMALLINT)  
BEGIN  
UPDATE employees SET salary = salary * 1.10 WHERE emp_id  
= empNum AND rating = 1;  
UPDATE employees SET salary = salary * 1.05 WHERE emp_id  
= empNum AND rating <> 1;  
END  
$$  
DELIMITER ;
```

How to Use This Procedure

```
CALL UPDATE_SAL('E12345', 1);
```

What Happens?

1. The procedure receives:
 - `empNum = 'E12345'` (employee ID).
 - `rating = 1` (performance rating).
2. It executes the first `UPDATE` statement because `rating = 1`:
 - Increases the salary of employee `'E12345'` by 10%.
3. The second `UPDATE` statement is skipped because `rating <> 1` is false.

Triggers

- Execute automatically on database events.
- Example:
Keeping an audit log whenever a new order is inserted.

Triggers

```
CREATE TRIGGER after_insert_order  
AFTER INSERT ON orders  
FOR EACH ROW  
INSERT INTO audit_log (order_id, action) VALUES (NEW.order_id,  
'Inserted');
```

User-Defined Functions (UDFs)

- Custom functions for reusable computations.
- Example:
Standardizing how full names are displayed.

User-Defined Functions (UDFs)

```
CREATE FUNCTION GetFullName(@first_name VARCHAR(50), @last_name  
VARCHAR(50))  
RETURNS VARCHAR(100)  
AS  
BEGIN  
    RETURN @first_name + ' ' + @last_name;  
END;
```

Window Functions

A window function makes a calculation across multiple rows that are related to the current row. For example, a window function allows you to calculate:

- Running totals
- 7-day moving averages
- Rankings

Window Functions

SELECT

```
    window_function() OVER(  
        PARTITION BY partition_expression  
        ORDER BY order_expression  
        window_frame_extent  
    ) AS window_column_alias  
FROM table_name
```

Aggregate window functions

SELECT

name, salary, AVG(salary) OVER (

PARTITION BY department_id

) AS avg_salary

FROM employees;

Value window functions

SELECT

```
name, salary, LEAD(salary) OVER (  
    ORDER BY salary  
) AS next_salary  
FROM employees;
```

Rank window functions

SELECT

```
name, salary, RANK() OVER (  
    ORDER BY salary DESC  
) AS salary_rank  
FROM employees;
```

Common Table Expressions (CTEs)

Temporary named result sets that can be referenced within a SELECT, INSERT, UPDATE, or DELETE Statement.

- CTEs can be used multiple times and for different purposes in the main query

Common Table Expressions (CTEs)

WITH EmployeeSalary AS (

SELECT

name, salary

FROM employees

WHERE salary > 50000

)SELECT *

FROM EmployeeSalary;

Common Table Expressions (CTEs)

WITH EmployeeSalary AS (

SELECT

name, salary

FROM employees

WHERE salary > 50000

)SELECT *

FROM EmployeeSalary;

Indexes

A data structure that provides quick access to data (the rows) optimizing the speed of the query. It is like a guide in the database.

- To create an index for a column in a table:

```
CREATE INDEX idx_department ON employees (department);
```

- To remove index:

```
DROP INDEX IF EXISTS idx_department;
```

Cursor

- The Cursor is a handle (name or a pointer) for the memory associated with a specific SQL statement.
- A cursor is basically an Area allocated by Oracle for executing the SQL Statements.
- Oracle Has two basic Types of Cursor:
 - Implicit Cursors
 - Explicit Cursors
- Use them when you:
 - Need to perform operations on each row (e.g., conditional updates or complex calculations).
 - Set-based operations (like UPDATE or INSERT with JOIN) are not feasible.

Implicit Cursors

- Automatically created by Oracle every time a DML statement like INSERT, UPDATE, DELETE, or SELECT INTO is executed, provided it's a single row query.
- Associated with explaining if any rows were affected by a query and their number.
- Example:

```
BEGIN
```

```
  UPDATE employees SET salary = salary * 1.10 WHERE department_id =  
  10;
```

```
  IF SQL%FOUND THEN
```

```
    DBMS_OUTPUT.PUT_LINE('Rows updated: ' || SQL%ROWCOUNT);
```

```
  END IF;
```

```
END;
```

Explicit Cursors

- Declared by the programmer to handle SQL queries (Select Statements) that return more than one row.
- Creation and manipulation of explicit cursors requires a number of manual steps:
 1. Declare a Cursor
 2. Open Statement
 3. FETCH Statement
 4. CLOSE Statement

Explicit Cursors

- Example:
DECLARE
CURSOR c1 IS SELECT employee_id, salary FROM employees
WHERE department_id = 20;
v_id employees.employee_id%TYPE;
v_sal employees.salary%TYPE;
BEGIN
OPEN c1;
LOOP
FETCH c1 INTO v_id, v_sal;
EXIT WHEN c1%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('ID: ' || v_id || ', Salary: ' || v_sal);
END LOOP;
CLOSE c1;
END;

Cursors Vs. Set-based SQL

Feature	Set-based SQL	Cursors
Speed	Faster (batch operations)	Slower (row-by-row)
Memory Usage	Efficient	More memory consumption
Complexity	Less code	More verbose
Control Over Row	Limited	Full

Questions & Answers



Thank you!